Quality assurance in Reusable Delivery

Plan the evidence before work starts. Give each bounded task one executor and record the result against the story's acceptance criteria.

Where it lives

Both delivery guides place QA in the story's **Execution checklist**. The coordinator chooses checks, timing and reviewers from the agreed criteria, risk and repository policy. The accountable human assigns the coordinator and owns acceptance policy and release decisions.

Bounded step	When and who	Evidence in the story
Plan	Before the first build step. Story coordinator.	Criteria, failure paths, checks and assigned executors.
Implement	After the outcome is agreed. One assigned writer.	Proposed revision and writer checks. AI steps name the planned model and reasoning.
Verify	On the proposed change. Assigned automation or manual verifier.	Command, runtime, tested revision, result and run link.
Review	After writer checks and before handoff or merge for material changes. Independent human or approved AI.	Findings tied to criteria, reproducible evidence and affected rechecks after repairs.
Decide	Before acceptance or release when policy requires it. Accountable human.	Decision, authority and any conditions.

When to adopt

New stories plan verification before implementation. **Migrated stories** preserve verified results and apply the plan to their next implementation step or reopened checks. Design and documentation stories check sources, links and criteria as applicable.

> The Karpathy loop stays the same. Select one eligible step, execute it, verify the result, record the evidence and select the next step.

Read the published QA checklist and workflow instructions

Evidence and rollout

The shipment Python prototype is the first consumer. Both guides share the QA decision rules. The optional CI workflow runs the consumer's repository-owned **check.py** with a read-only token.

What the pilot demonstrated

Check or measurement	Observed result
Local shipment checks	**22 tests passed** on Python 3.14.8. A deliberately failing test and empty test discovery both returned a failure.
GitHub Actions pilot	**Passed on Python 3.11 and 3.14** in the shipment draft pull request. The workflow records the tested revision and runtime.
Proposed size caps	Zero hits in either Python snapshot at 50 lines per function or 250 per file. Longest production function was 24 lines under the measured count.
Proposed two-parameter cap	Production functions flagged increased from **9 to 10** after the repair. The source-isolation fix added a required source identity parameter.

Size measurements count nonblank, non-comment lines in Python. Parameter counts exclude self or cls. This was a static measurement of the two shipment revisions.

Adoption decisions

Enforce relevant behavior checks in CI. Use the same repository command for local feedback. Add lightweight editor or agent hooks when they give timely feedback. Treat length, complexity and duplication as review signals calibrated to the repository.

Evaluate independent review by evidence. Compare tools on the same revision and context. Record verified findings, misses, review time and cost. The cited four-defect TypeScript benchmark is a single trial. The local checks use no model inference.

Rollout status. The guide and reusable workflow are published. Shipment is a draft pilot with passing CI. A controlled CI failure remains to be observed before making the check required. The CodeRabbit comparison remains pending GitHub verification.

Published validation record

Shipment CI pilot pull request

Python 3.11 and 3.14 job results

Claude Code hook behavior and limits

Original four-defect TypeScript benchmark